Lennard-Jones molecular dynamics at GPU speed from pure Python: an optimization ladder from NumPy to Triton, Warp, Numba and CUDA C

Walid Hemati
Independent researcher

October 2026

Abstract

We ask whether a molecular-dynamics (MD) code written entirely in Python can use a GPU as efficiently as hand-written CUDA. `pymd` is a Lennard-Jones (LJ) and second-moment (Gupta/EAM) MD engine whose time-stepping logic is written once and whose hot loops exist as interchangeable backends: NumPy, PyTorch (eager and `torch.compile`), Triton, NVIDIA Warp, Numba-CUDA and hand-written CUDA C compiled at run time through CuPy/NVRTC. All backends share GPU cell lists built by counting sort, Verlet neighbor lists with a skin, and spatial atom sorting. On the LAMMPS `in.lj` melt benchmark, scaled up to four million atoms on a laptop RTX 3070, the fastest variant (cupy-cuda-c/full-fused) reaches 508 million atom-steps per second at $N = 1{,}048{,}576$, i.e. 49% of the measured copy bandwidth under a minimal-traffic model. The Python kernel languages run the same algorithm at 92% (Triton), 82% (Warp) and 84% (Numba) of the throughput of the CUDA C reference at $N = 2{,}048{,}000$. Correctness is established against a float64 NumPy oracle (relative force error $\leq 9.6e-06$ in single precision), the LAMMPS reference log of `in.lj`, NIST reference simulation data, the Johnson–Zollweg–Gubbins equation of state, and the author's earlier C++ code `yamd`, whose forces `pymd` reproduces to $1e-14$. We also build LAMMPS (GPU and KOKKOS packages) and HOOMD-blue and run them on the same GPU on the same, cross-verified system.

1 Introduction

Classical MD of simple liquids is the canonical short-range particle workload: each step gathers the positions of ∼50–100 neighbors per atom, evaluates a cheap pair function, and accumulates forces. Production GPU codes such as LAMMPS' GPU and KOKKOS packages [1, 2] and HOOMD-blue [3–5] implement this in CUDA C++. In parallel, a family of Python GPU programming tools has matured: Triton [6], NVIDIA Warp [7], Numba [8], CuPy [9] and PyTorch [10] with its compiler `torch.compile` [11]. This paper measures how close each of them comes to the hardware limit on the same problem, with the same algorithms, on the same GPU.

`pymd` is the Python successor of `yamd` [12], the author's C++/Eigen MD code (LJ direct summation, velocity Verlet, Berendsen thermostat, cell-based neighbor lists, and a Gupta potential for gold clusters). `pymd` mirrors `yamd`'s features and test cases and adds periodic boundaries, GPU neighbor search and a Langevin thermostat. Contributions:

- An MD engine with one integrator and a registry of 23 backend/variant combinations spanning seven programming models, all validated against one float64 oracle (sections 4 and 5).

- A minimal memory-traffic and flop model per atom-step for Verlet-list MD, used to report every throughput as a fraction of the measured copy bandwidth and FMA peak (section 4.1).
- An optimization ladder with ablations of spatial sorting (none, cell order, Morton order), full versus half (atomic) neighbor lists, kernel fusion, and precision (fp32, mixed, fp64) (section 6).
- Validation against the LAMMPS `in.lj` reference output, NIST SRSW Monte Carlo data [13], the JZG equation of state [14], and a same-machine cross-check and head-to-head with the C++ code `yamd` (sections 5 and 6.2).
- A same-GPU head-to-head with LAMMPS (GPU and KOKKOS packages, double and single precision) and HOOMD-blue (conda-forge double-precision build and a single-precision source build), after checking that all codes integrate an identical initial state to the same trajectory (section 6.1).

2 Background and related work

GPU MD with Verlet lists was established by Anderson et al. [3], who already used space-filling-curve (Hilbert) reordering of particles for memory locality, and for LAMMPS by Brown et al. [1]; Glaser et al. studied strong scaling of HOOMD-blue across GPUs [4]. Python front ends are common (HOOMD-blue v2+ [5], OpenMM [15]), but their kernels are C++/CUDA. JAX MD [16] expresses MD in a differentiable array language. Our question is different: what fraction of the GPU's capability do the Python *kernel* languages themselves reach? We answer with the roofline method [17]. Textbook background on the algorithms we use is in Allen & Tildesley [18] and Frenkel & Smit [19].

3 Method

Model. Atoms interact through the LJ potential $u(r) = 4\epsilon[(\sigma/r)^{12} - (\sigma/r)^6]$ truncated at r_c (optionally shifted so that $u(r_c) = 0$), in reduced units $\sigma = \epsilon = m = k_B = 1$, with periodic boundaries and the minimum-image convention. For comparisons with full-LJ data we add the standard long-range corrections [18]. Metals use the second-moment tight-binding (Gupta) potential $E_i = \sum_j A e^{-p(r_{ij}/r_0-1)} - \left(\sum_j \xi^2 e^{-2q(r_{ij}/r_0-1)}\right)^{1/2}$ [20–22] with `yamd`'s gold parameters ($A = 0.2061\,\mathrm{eV}$, $\xi = 1.790\,\mathrm{eV}$, $p = 10.229$, $q = 4.036$, $r_0 = 4.079/\sqrt{2}\,\text{Å}$, cutoff $10\,\text{Å}$), evaluated in two passes (density, then force).

Integration. Velocity Verlet [23] with the two half kicks of consecutive steps fused into one kick-drift kernel; the fused variants use the equivalent leapfrog form so that force, kick and drift happen in a single kernel per step (positions are double-buffered). Thermostats: Berendsen velocity rescaling [24] (as in `yamd`) and Langevin dynamics with the BAOAB splitting [25].

Neighbor search. At each rebuild, positions are wrapped into the box and binned into cells of edge $\geq r_c + r_s$ (r_s: skin). A counting sort (atomic histogram, exclusive scan, atomic-slot scatter) yields cell start offsets; by default the atom arrays are then physically permuted into cell order, so that neighbors in space are neighbors in memory. The cell key is either row-major or a Morton (Z-order) interleave of the cell coordinates. A Verlet list of all j with $|r_{ij}| < r_c + r_s$ is then built over the 27-cell stencil with one thread per atom and stored neighbor-major (`nl`$[kN+i]$) for coalesced reads;

an overflow flag triggers a resize and rebuild. The list is rebuilt every n_{every} steps, unconditionally (LAMMPS "`check no`") or only if some atom moved more than $r_s/2$ ("`check yes`").

Benchmark problem. We use the LAMMPS `bench/in.lj` melt [26]: FCC lattice at $\rho^* = 0.8442$, initial $T^* = 1.44$, $r_c = 2.5\sigma$, $r_s = 0.3\sigma$, list rebuilt every 20 steps without a check, NVE, $\Delta t = 0.005\tau$. The original has 32,000 atoms; we scale it from 4,000 to 4,000,000. Atom order is randomly shuffled at $t = 0$ so that sorting has to earn its benefit. The metric is atom-steps per second (LAMMPS' "atom-step/s"); we also give τ/day and, with the argon mapping $\sigma = 0.3405\,\text{nm}$, $\epsilon/k_B = 119.8\,\text{K}$, $m = 39.948\,\text{u}$ ($\tau \approx 2.156\,\text{ps}$), ns/day.

4 Implementation: the optimization ladder

A backend implements nine primitives (rebuild, forces with or without energy/virial, kick-drift, max displacement, kinetic energy, velocity scaling, OU noise, up/download); the driver in `engine.py` implements Verlet, the rebuild schedule and the thermostats once.

NumPy (float64, CPU): cell list via `argsort`, padded cell table, vectorized candidate generation per stencil offset, half pair list, `np.bincount` scatter. This is also the correctness oracle (together with an $O(N^2)$ direct sum).

PyTorch eager a padded atom-major (N, M) neighbor matrix and a *gather* formulation, `x[nl]`, processed in chunks to bound the (N, M) intermediates.

`torch.compile` the same code compiled by TorchInductor. Writing the gather per Cartesian component (structure of arrays) let Inductor fuse gather, pair math and reduction into one generated Triton kernel; the array-of-structures version was $\sim 3\times$ slower.

Triton hand-written kernels: block of 128 atoms per program, a dynamic loop to the block's maximum neighbor count, constexpr switches for energy, fusion and half lists. Rebuild-time glue (key sort, permutation) uses PyTorch.

Warp a hand-written counting-sort cell list, list build and force kernels, and two variants using Warp's built-in `HashGrid`: `hashgrid` (no Verlet list; grid rebuilt and queried every step) and `hashgrid-verlet` (grid used only to build the list). With a cubic box and $\lfloor L/r \rfloor$ cells per side, the hash wrap coincides with the periodic wrap.

Numba-CUDA the same kernel set in `@cuda.jit`, including a single-block scan.

CuPy CUDA C the hand-written reference, compiled by NVRTC, with `float4` positions and templated precision. Variants: `full`, `full-fused`, `half-atomic` (Newton's third law with `atomicAdd`), `full-tpa2/4` (two or four threads per atom, warp-shuffle reduction), `full-morton`, `full-nosort`, `full-mixed` (fp64 positions, velocities and integration; fp32 pair math and per-atom sums), `full-fp64`, and a shared-memory tiled $O(N^2)$ `allpairs`.

4.1 Traffic and flop model

For one Verlet-list step the compulsory DRAM traffic per atom is: read $\mathbf{x}_i$, read and write $\mathbf{v}_i$, write $\mathbf{x}_i$ (12 words) and read the stored list, 4 bytes per entry: $B = 48 + 4\,n_\ell$ bytes in fp32 ($96 + 4n_\ell$ with fp64 storage), where n_ℓ is the measured mean list length ($\approx$75 for a full list at $r_c + r_s = 2.8\sigma$, half that for a half list). Neighbor positions are assumed to come from cache and the amortized

rebuild traffic is ignored, so the effective bandwidth $R \cdot B$ is a lower bound and never inflated. Each list entry costs 20 flop (difference, minimum image, r^2; FMA = 2 flop) and each pair inside r_c (a fraction $(r_c/(r_c+r_s))^3$ of entries) 13 more. The arithmetic intensity is ≈6 flop/byte, well left of the ridge point (15.5 TFLOP/s / 361 GB/s), so the bandwidth roofline is the relevant ceiling. Ceilings: 361 GB/s device-to-device copy (`torch copy_`, read+write), 448 GB/s specification, and 15.5 TFLOP/s from our fp32 FMA microbenchmark.

5 Validation

Forces. For a liquid configuration (N = 4,000, ρ^* = 0.8442, T^* = 0.722) every variant is compared with the float64 NumPy direct sum (fig. 1a). All single-precision variants agree to $\max_i |\Delta F_i| / \max_i |F_i|$ between 9.0e-06 and 9.6e-06; mixed precision reaches 9.0e-06 and fp64 6e-16. The CPU tests port `yamd`'s GoogleTest cases (finite-difference LJ and Gupta forces, constant-force Verlet, Berendsen relaxation, neighbor-list cases) and run in CI.

LAMMPS `in.lj`. Step 0 is deterministic: on the GPU we obtain $E_{\mathrm{pair}} = -6.7733674$ and $P = -5.0197081$ versus LAMMPS' −6.7733681 and −5.0197073 (the float64 NumPy path matches all digits). After 100 steps the trajectories depend on the random velocities, so we average 8 seeds drawn, like LAMMPS' default, from a uniform distribution: $T = 0.7586 \pm 0.0030$, $E_{\mathrm{pair}} = -5.7602 \pm 0.0044$, $E_{\mathrm{tot}} = -4.6224 \pm 0.0001$, $P = 0.1942 \pm 0.0174$ (mean ± seed-to-seed standard deviation), against LAMMPS' single trajectory 0.7574531, −5.7585055, −4.6223613, 0.20726105. The half-list length is 37.554 (LAMMPS: 37.588531) with 5 rebuilds (LAMMPS: 5). (With Gaussian velocities the step-100 pressure is systematically higher, a reminder that the 100-step melt is not equilibrated.)

Energy conservation. NVE runs of a $N = 4000$ liquid ($\rho^* = 0.8442$, $T^* \approx 0.72$, shifted potential, list checked every step) show the expected Δt^2 scaling of the energy fluctuation for $\Delta t \geq 0.003$ (fitted exponent 1.83, fig. 1b); at smaller steps the fluctuation levels off, as expected for a potential whose force is discontinuous at r_c. Over $500\,\tau$ at $\Delta t = 0.005$ the fitted linear drifts are 2.1e-07 (fp32), 5.0e-07 (mixed) and 3.5e-07 (fp64) ϵ/τ per atom (fig. 1c). These are all small compared with the fluctuation and of similar size for the three precisions; repeated runs show energy wander of $\sim 10^{-4}\epsilon$ per atom over 200τ for every precision, so on this time scale the error budget is set by time discretization and the force cutoff, not by single-precision round-off.

Structure. Figure 2 shows $g(r)$ of the dense liquid $\rho^* = 0.8442$, $T^* = 0.722$ ($r_c = 3\sigma$, Langevin NVT). The first peak is at $r = 1.085\sigma$ with height 3.031. We did not find tabulated literature $g(r)$ values for this state that we could verify (Verlet's 1968 tables [27] were not accessible to us), so we test $g(r)$ through thermodynamic consistency instead: the energy route $U/N = 2\pi\rho \int u\, g\, r^2 \mathrm{d}r = -5.815$ equals the direct average -5.816 ± 0.003, the virial route gives $P = 0.627$ versus 0.619 ± 0.014, and the energy route with the full potential, -6.078, agrees with the JZG equation of state [14], -6.088.

Equation of state. At the 10 liquid NIST SRSW Monte Carlo state points ($T^* = 0.85, 0.90$, $\rho^* = 0.776$–0.90, $N = 500$, $r_c = 3\sigma$ + long-range corrections [13]) our Langevin NVT averages agree with NIST to within 0.06% in energy and 0.023 in pressure (largest deviation 1.6 combined standard errors), and both agree with JZG (fig. 1d). We transcribed the 32 JZG coefficients from NIST's `teqp`; they reproduce the NIST data, which is an independent check of the transcription.

EAM gold. For a perturbed Mackay icosahedron [28] of 923 Au atoms the GPU fp32 Gupta forces match float64 to 1.9e-05, and NVE with $\Delta t = 1$ fs conserves energy to 2.5e-06 eV/atom over 2 ps.

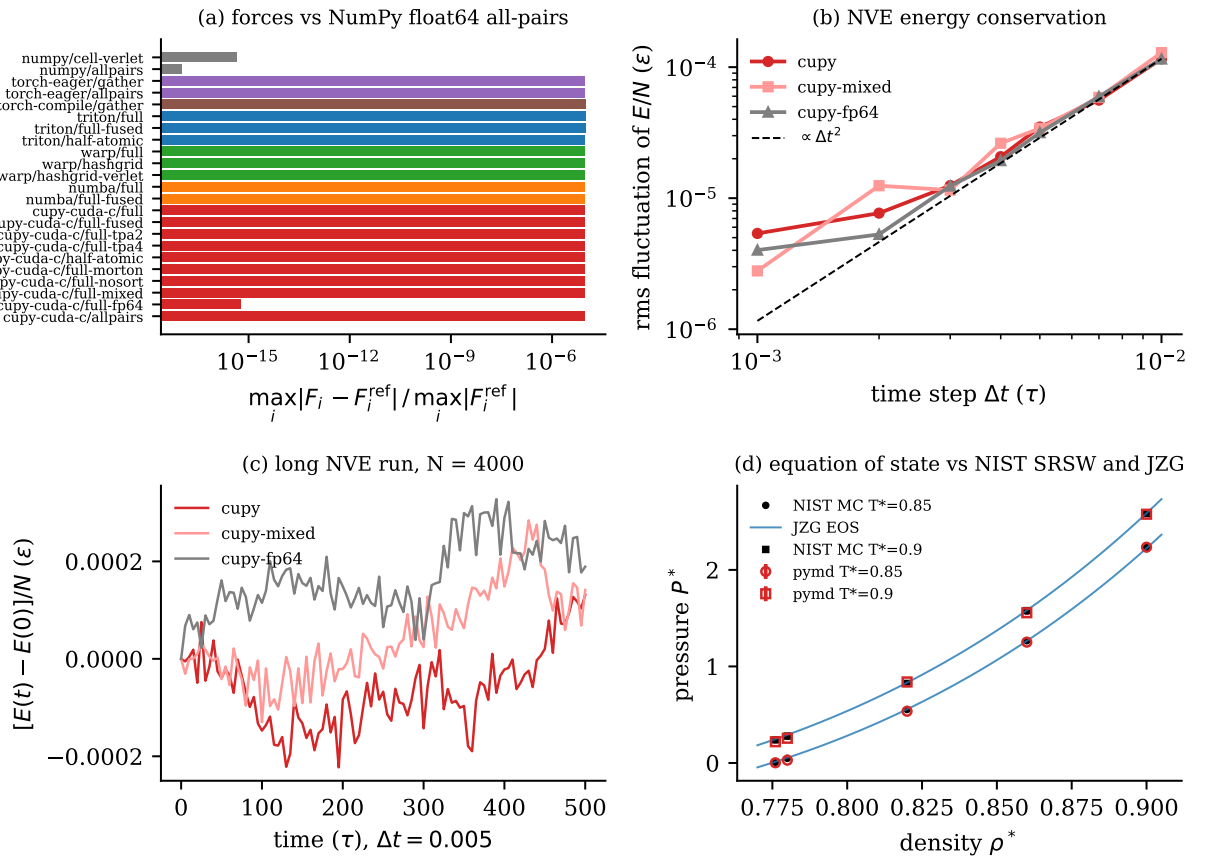


Figure 1: Validation. (a) Force error of every backend/variant versus the float64 oracle. (b) Energy fluctuation versus time step. (c) Energy drift in a long NVE run for three precisions. (d) Pressure versus NIST SRSW Monte Carlo data and the JZG equation of state.

6 Performance

All numbers are medians of five timed repeats of at least 0.5 s each after warm-up (JIT and compilation excluded), with the GPU synchronized around the timed region. Figure 3 shows throughput versus system size; fig. 4 the roofline and the fraction of copy bandwidth at $N = 2{,}048{,}000$.

The ladder. At $N = 2{,}048{,}000$ the best variant per backend reaches (Matom-steps/s, % of copy bandwidth): Numba 403.8 (39%), Warp 376.6 (36%), Triton 437.0 (42%) and CUDA C 488.0 (47%). The array rungs stop at smaller N on the 8 GB GPU: PyTorch eager reaches 7.1 Matom-steps/s ($N = 500{,}000$) and `torch.compile` 51.0 ($N = 1{,}048{,}576$, 5%). The NumPy CPU baseline runs at 0.21 Matom-steps/s at $N = 4000$. The array-programming rungs are limited by the neighbor-list rebuild, which materializes candidate arrays; the kernel languages share the same algorithm; we attribute their remaining differences to code generation details such as vector versus scalar position

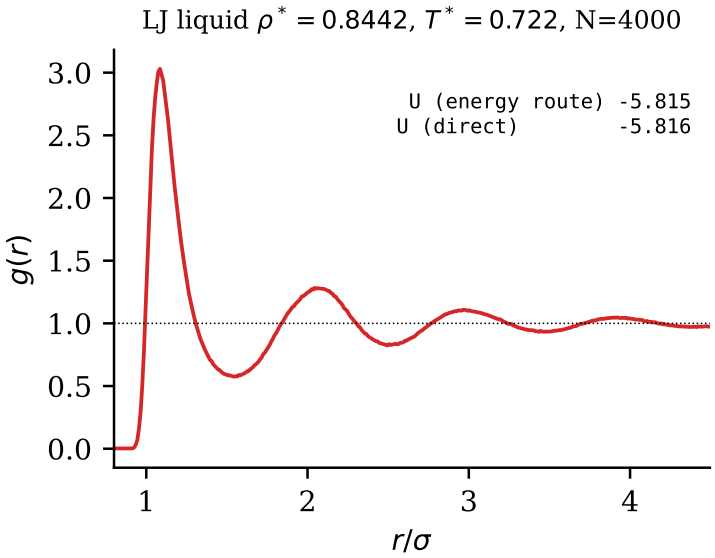


Figure 2: Radial distribution function of the LJ liquid at $\rho^* = 0.8442$, $T^* = 0.722$.

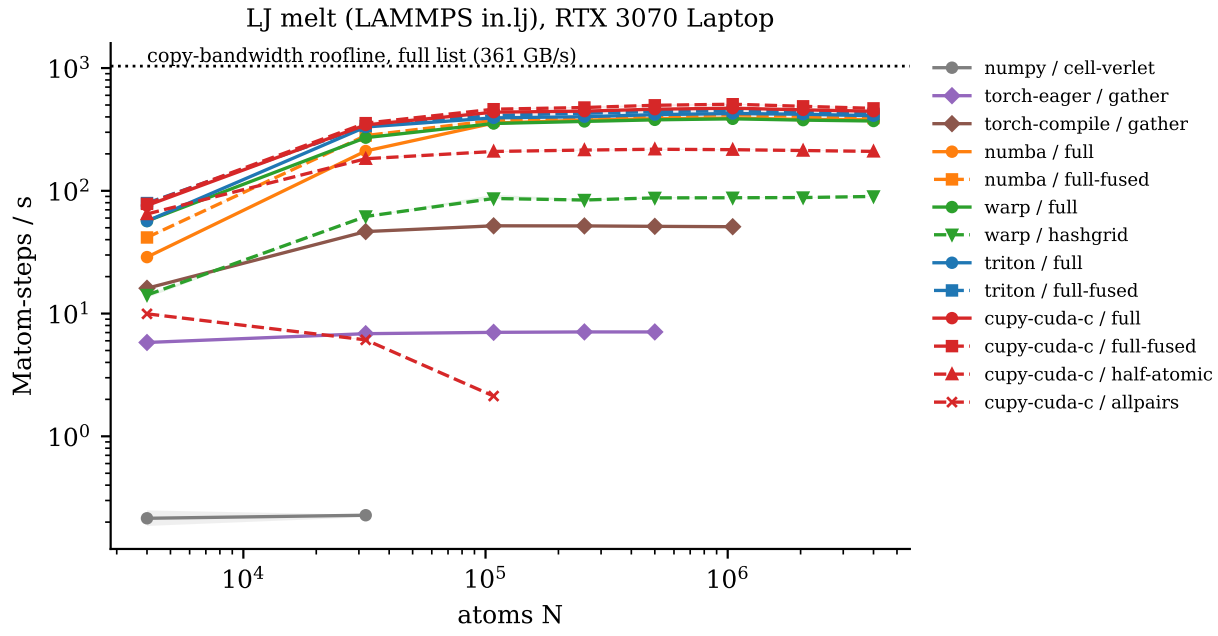


Figure 3: Throughput on the `in.lj` melt versus atoms (bands: min–max of 5 repeats). Dotted: the copy-bandwidth roofline of the minimal traffic model for a full list.

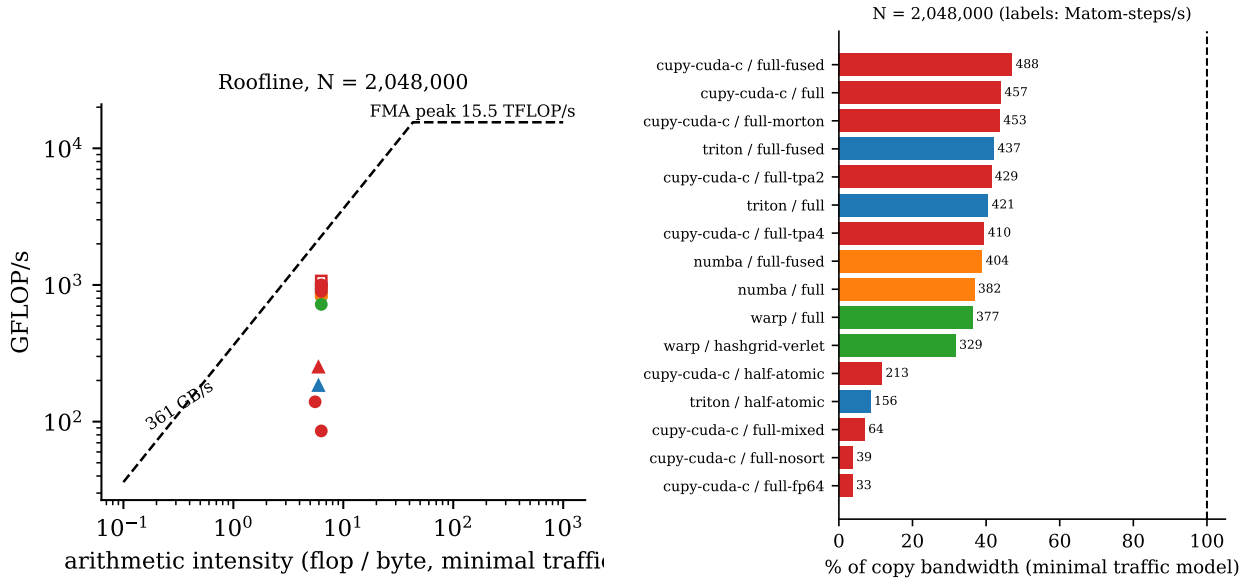


Figure 4: Left: roofline (minimal traffic model) at $N = 2{,}048{,}000$; filled markers unfused, open markers fused kernels. Right: fraction of the 361 GB/s copy bandwidth.

loads and per-launch Python overhead at small N (we did not profile these in depth). The best overall is cupy-cuda-c/full-fused with 508 Matom-steps/s at $N = 1{,}048{,}576$ (209,111 τ/day, 451 ns/day for argon), 49% of the copy and 39% of the specification bandwidth under our model.

Ablations (fig. 5, CUDA C, $N = 2{,}048{,}000$, relative to `full`): no spatial sorting 0.09×, Morton order 0.99× (row-major cell order is already local at this cell size), half list with atomics 0.47× (fewer list bytes but three float atomics per pair), fused kernel 1.07×, two and four threads per atom 0.94× and 0.90×, mixed precision 0.14× and fp64 0.07× (the consumer GPU executes fp64 at a small fraction of the fp32 rate).

6.1 Same-GPU head-to-head with LAMMPS and HOOMD-blue

Published rates of other codes are for other hardware, densities, cutoffs and precisions, so we measure two established GPU MD codes on the same laptop GPU instead. LAMMPS 30 Sep 2026 (`stable_30Sep2026`) [2] is built with CMake and `nvcc` 13.4 for `sm_86` as a serial binary (MPI stubs, no OpenMP) in four flavours: the GPU package [1] with `GPU_PREC=mixed` and `single`, and the KOKKOS package (Kokkos 5.2 [29], CUDA back end) with `KOKKOS_PREC=double` and `single`. Each runs with its default package settings (`-sf gpu -pk gpu 1`; `-k on g 1 -sf kk`, i.e. a full list with `newton off`); for KOKKOS we also time a half list with `newton on`. HOOMD-blue 7.2.0 [5] is the conda-forge GPU build, which is compiled with a 64-bit `LongReal`, so its pair forces are evaluated in float64; for a like-for-like comparison we also build the same release from source with `HOOMD_LONGREAL_SIZE=32` (a one-line type fix in its GSD writer is needed to compile). All codes run the `in.lj` problem of section 3 with the same skin and the same unchecked rebuild every 20 steps (HOOMD: `nlist.Cell(buffer=0.3, rebuild_check_delay=20, check_dist=False)`). LAMMPS creates its own lattice and velocities; HOOMD and `pymd` start from `pymd`'s shuffled lattice. LAMMPS rates are N times steps over LAMMPS' own loop time of each of 5 timed `run` commands after a warm-up run (setup excluded, as in its `Performance` line); HOOMD rates come

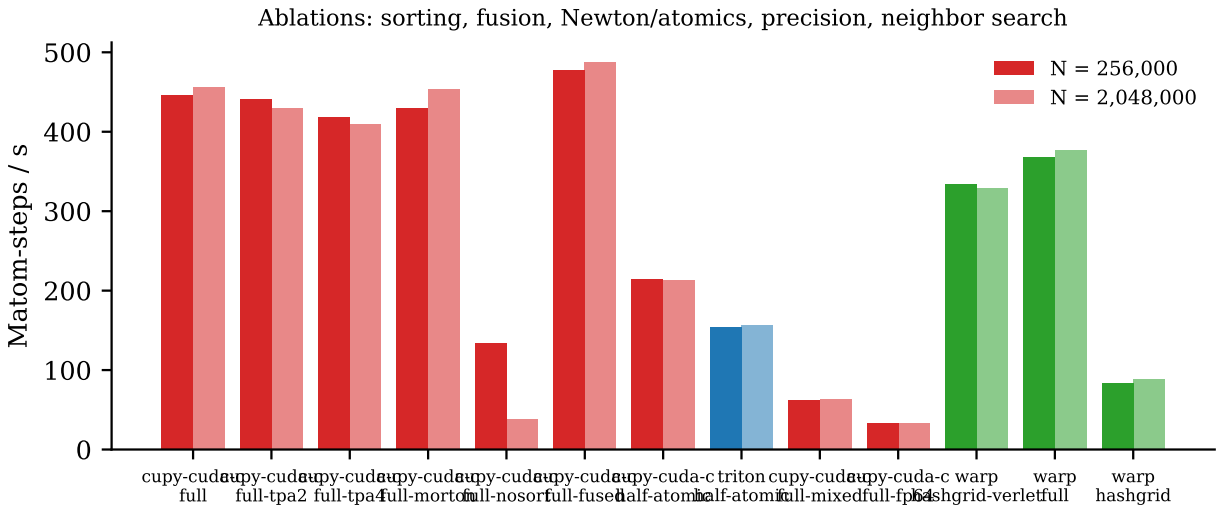


Figure 5: Ablations at two sizes.

from the wall time of `Simulation.run` after its autotuner has finished (up to 600 s of tuning per size; it finished at every N for the single-precision build and at all but the two largest N for the double-precision build). The `pymd` variants are timed in the same session.

Same system. Before timing, every build integrates an identical initial state (32,000 atoms, `pymd`'s lattice and velocities, read by LAMMPS through `read_data` and by HOOMD as a snapshot) for 100 steps. At step 0 the potential energy of every build agrees with `pymd` in float64 to $\leq 1\times10^{-7}$ (LAMMPS) and $\leq 9\times10^{-8}$ (HOOMD), and the pressure to $\leq 3\times10^{-7}$ and $\leq 2\times10^{-7}$; the float32 builds set these maxima. After 100 steps all LAMMPS builds agree to $\leq 6\times10^{-7}$ in energy and $\leq 6\times10^{-5}$ in pressure, the same level as `pymd`'s own float32 variant (4×10^{-7}, 3×10^{-5}). HOOMD differs by up to 6×10^{-6} and 3×10^{-4}. This comes from the benchmark's rebuild schedule, not from the force field: with rebuilds every 20 steps and no displacement check, some pairs that cross the cutoff between rebuilds are missed, and which ones depends on exactly when a code rebuilds. With the neighbor list kept complete (rebuild checks every step), HOOMD agrees with `pymd` to 3×10^{-8} in energy.

Throughput. At $N = 4{,}000{,}000$ (fig. 6) the best `pymd` variant (`cupy-cuda-c/full-fused`) runs at 470.5 Matom-steps/s and the best Python kernel language (`triton/full-fused`) at 426.2. The fastest established code is HOOMD-blue `gpu-single` (single precision) at 364.3 Matom-steps/s, so `pymd`'s best variant reaches 1.29× and its best Python-DSL variant 1.17× its throughput. HOOMD-blue runs at 364.3 Matom-steps/s in single precision and 30.4 in the float64 conda build. LAMMPS KOKKOS reaches 103.2 (`kokkos-single-half`) in single and 58.1 (`kokkos-double-half`) in double precision, the GPU package 50.4 (`gpu-single`), and LAMMPS on one CPU core 2.65 at $N = 108{,}000$.

Why the codes differ. (i) *Precision.* Consumer Ampere GPUs execute float64 at a small fraction of the float32 rate, so every float64 build (HOOMD from conda-forge, KOKKOS by default) is limited by double-precision arithmetic; `pymd`'s own float64 variant (33 Matom-steps/s at $N =$ 2,048,000, fig. 5) is in the same range. The float64 defaults are the right choice for data-centre GPUs;

here single precision is 12.0× faster for HOOMD-blue and 1.8× for KOKKOS. (ii) *KOKKOS in single precision.* `KOKKOS_PREC=single` makes the pair arithmetic float32, but the neighbor-list build still computes pair distances in float64 (the kernels in `npair_kokkos.cpp` cast positions to `double`), and on GPUs it bins atoms with a bin edge of a full neighbor cutoff. In a development profile at $N = 256{,}000$ (Nsight Systems, shared GPU) this build, run every 20 steps by the benchmark, took most of KOKKOS' GPU time, more than the force kernel. A half-cutoff bin size and `neigh/thread on` did not help in development runs, so the benchmark keeps the defaults plus the half-list variant. (iii) *GPU package.* Only the pair force and the neighbor list run on the GPU (with binning on the host); time integration (`fix nve/gpu`) runs on the CPU, and positions and forces cross PCIe every step. The package is designed for several MPI ranks sharing one GPU to hide this, which a serial build cannot do. (iv) *HOOMD-blue in single precision* keeps all data on the GPU, uses full neighbor lists and autotunes its launch parameters, as `pymd` does, and is the closest counterpart. Its remaining difference to `pymd` comes from implementation details (list layout, number of kernels per step, sorting policy) that we did not profile in depth. At small N, per-step launch overheads weigh more for codes that launch several kernels per step.

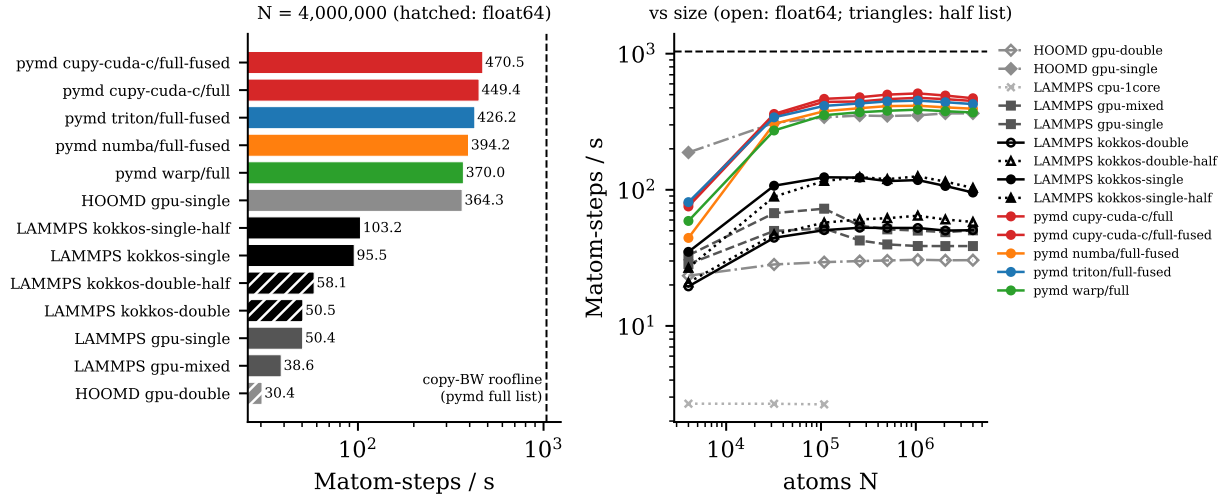


Figure 6: Same-GPU comparison on the `in.lj` melt: `pymd` (backend colors) versus LAMMPS 30 Sep 2026 (GPU and KOKKOS packages, one CPU core) and HOOMD-blue 7.2.0 (black and grey). Hatched bars and open markers: float64 arithmetic. Dashed: copy-bandwidth roofline of `pymd`'s full-list traffic model (it does not apply to the other codes' layouts).

Published numbers. For context only: Anderson et al. report 0.0778 μs per particle and step for an LJ liquid at $\rho^* = 0.382$, $r_c = 3\sigma$ (43.2 neighbors per particle) on a GeForce 8800 GTX in 2008 [3], and Anderson, Glaser and Glotzer report 275×10^6 particle time steps per second for $N = 64{,}000$ at the same density, $k_BT = 1.2$, on a single V100 GPU [5]. The archived LAMMPS benchmark page lists 2.35 s for 100 steps of the 1-million-atom `in.lj` problem on one Tesla M2090 (2012) [30], i.e. about 43 Matom-steps/s, and the LAMMPS reference log of `in.lj` reports 3.752 Matom-step/s on one CPU core [26].

6.2 Head-to-head with the C++ predecessor `yamd`

`yamd`'s MPI-free library sources compile unchanged with `g++ -O3 -march=native` (its meson build requires MPI, which is not installed, and an Eigen subproject that is incomplete in the repository; we use the Eigen 3.4.0 tarball from its package cache, whose hash matches the wrap file). Both codes then run identical open-boundary systems on the same machine (fig. 7). Forces agree to 1e-14 (LJ) and 6e-15 (EAM) absolute. For an FCC block of 32,000 atoms, single-threaded `yamd` (neighbor list rebuilt every step without skin, as `yamd` does) runs at 0.31 Matom-steps/s; the same algorithm in NumPy runs at 0.059 (5× slower than C++) and on the GPU (CUDA C) at 69.2 (220× faster). With `pymd`'s own skin and rebuild schedule the GPU reaches 383.5. For a 3,871-atom gold cluster `yamd` runs EAM at 0.125 and the GPU at 6.4 Matom-steps/s. These are apples-to-oranges comparisons (one CPU core versus a GPU; different neighbor-list policies) included to document the lineage; the CPU is an AMD Ryzen 7 5800H with Radeon Graphics, otherwise idle during the measurement.

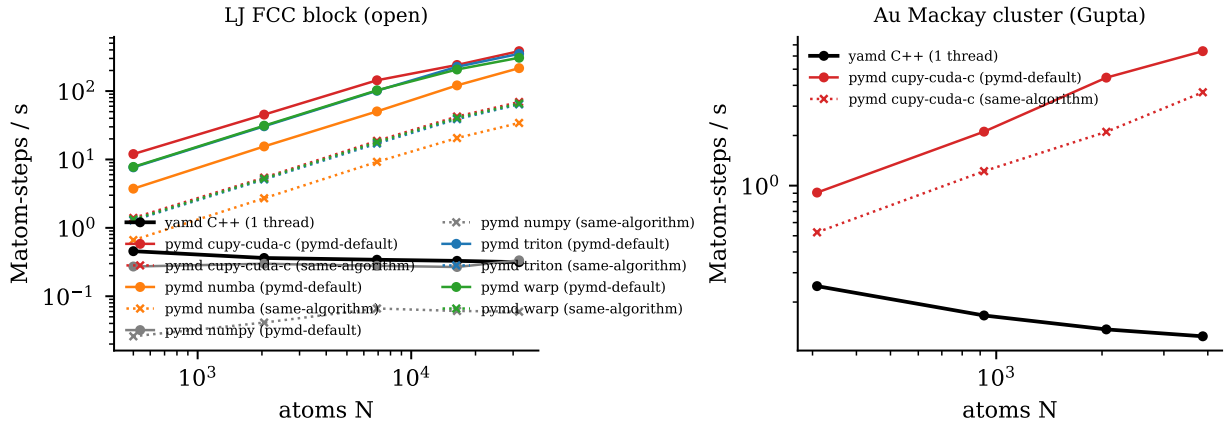


Figure 7: Same-machine comparison with the C++ code `yamd` (single thread). "same-algorithm": neighbor list rebuilt every step without skin, as in `yamd`; "pymd-default": skin and rebuild schedule of `pymd`.

7 Discussion

Where the time goes. With sorted atoms the force kernel streams the neighbor index list and hits cache for neighbor positions; the list bytes dominate the minimal model. The remaining gap to the bandwidth ceiling is latency of dependent gathers (index, then position) with one thread per atom; assigning two or four threads to each atom (the `full-tpa` variants) raises memory-level parallelism but adds a shuffle reduction and partially idle lanes. Half lists halve the index traffic but the atomic scatter costs more than it saves on this GPU.

Threats to validity. (i) A single laptop GPU; laptop GPUs throttle under sustained load, and clocks were not locked. (ii) Development timings were taken on a GPU shared with other jobs; the published table comes from a serial run on an idle GPU. (iii) CUDA C is compiled by NVRTC through CuPy, not `nvcc`; code generation is the same compiler back end but flags differ. (iv) The traffic model excludes neighbor-position gathers and rebuilds, so percentages are lower bounds on achieved bandwidth, and they are not comparable across codes that use different list layouts. (v)

Published numbers of other codes are for different densities, cutoffs, precisions and hardware; this is why section 6.1 measures LAMMPS and HOOMD-blue on our GPU. (vi) The `yamd` comparison mixes CPU and GPU. (vii) We built and ran LAMMPS and HOOMD-blue ourselves with their default settings plus the variants listed; experts may tune them further (e.g. several MPI ranks per GPU for the LAMMPS GPU package, which our serial build cannot use), and the source build of HOOMD-blue in single precision is not an official binary.

8 Conclusion

Python kernel languages are not a compromise for short-range MD: on the LAMMPS LJ benchmark Triton, Warp and Numba reach a large fraction of a hand-written CUDA C implementation of the same algorithm, and the best variant uses 49% of the copy bandwidth under a conservative model, with validation against LAMMPS, NIST and an equation of state. The array-programming rungs (NumPy, PyTorch) are easy to write but are an order of magnitude or more behind, mostly because neighbor search does not map well onto dense array operations.

Code availability

`pymd` is MIT-licensed at https://github.com/walidhemati/pymd. `bench/run.py -full` regenerates the benchmark JSON, `pymd validate` the validation JSON, `figures/make_figures.py` all figures and the numbers in this paper.

References

[1] W. Michael Brown, Peng Wang, Steven J. Plimpton, and Arnold N. Tharrington. Implementing molecular dynamics on hybrid high performance computers – short range forces. *Computer Physics Communications*, 182(4):898–911, 2011. doi: 10.1016/j.cpc.2010.12.021.

[2] Aidan P. Thompson, H. Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, et al. LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022. doi: 10.1016/j.cpc.2021.108171.

[3] Joshua A. Anderson, Chris D. Lorenz, and A. Travesset. General purpose molecular dynamics simulations fully implemented on graphics processing units. *Journal of Computational Physics*, 227(10):5342–5359, 2008. doi: 10.1016/j.jcp.2008.01.047.

[4] Jens Glaser, Trung Dac Nguyen, Joshua A. Anderson, Pak Lui, Filippo Spiga, et al. Strong scaling of general-purpose molecular dynamics simulations on GPUs. *Computer Physics Communications*, 192:97–107, 2015. doi: 10.1016/j.cpc.2015.02.028.

[5] Joshua A. Anderson, Jens Glaser, and Sharon C. Glotzer. HOOMD-blue: A Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations. *Computational Materials Science*, 173:109363, 2020. doi: 10.1016/j.commatsci.2019.109363.

[6] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL '19)*, pages 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

[7] Miles Macklin. Warp: A high-performance python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), March 2022. URL https://github.com/NVIDIA/warp.

[8] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 1–6. ACM, 2015. doi: 10.1145/2833157.2833162.

[9] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[10] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, et al. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[11] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 929–947. ACM, 2024. doi: 10.1145/3620665.3640366.

[12] Walid Hemati. yamd: molecular dynamics in C++. https://github.com/walidhemati/yamd. Commit 858345e, accessed 2026-10-09.

[13] V. K. Shen, D. W. Siderius, W. P. Krekelberg, and H. W. Hatch, editors. *NIST Standard Reference Simulation Website*. NIST Standard Reference Database Number 173. National Institute of Standards and Technology, Gaithersburg MD, 20899. doi: 10.18434/T4M88Q. URL http://doi.org/10.18434/T4M88Q. retrieved 2026-10-09.

[14] J. Karl Johnson, John A. Zollweg, and Keith E. Gubbins. The Lennard-Jones equation of state revisited. *Molecular Physics*, 78(3):591–618, 1993. doi: 10.1080/00268979300100411.

[15] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlles R. A. Abreu, Stephen E. Farr, et al. OpenMM 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, 128(1):109–116, 2024. doi: 10.1021/acs.jpcb.3c06662.

[16] Samuel Schoenholz and Ekin Dogus Cubuk. JAX MD: A framework for differentiable physics. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 11428–11441. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/83d3d4b6c9579515e1679aca8cbc8033-Paper.pdf.

[17] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[18] Michael P. Allen and Dominic J. Tildesley. *Computer Simulation of Liquids*. Oxford University Press, Oxford, 2nd edition, 2017. ISBN 9780198803195. doi: 10.1093/oso/9780198803195.001.0001.

[19] Daan Frenkel and Berend Smit. *Understanding Molecular Simulation: From Algorithms to Applications*. Academic Press, San Diego, 2nd edition, 2002. ISBN 9780122673511. doi: 10.1016/B978-0-12-267351-1.X5000-7.

[20] F. Ducastelle. Modules élastiques des métaux de transition. *Journal de Physique*, 31(11-12):1055–1062, 1970. doi: 10.1051/jphys:019700031011-120105500.

[21] Raju P. Gupta. Lattice relaxation at a metal surface. *Physical Review B*, 23(12):6265–6270, 1981. doi: 10.1103/PhysRevB.23.6265.

[22] Fabrizio Cleri and Vittorio Rosato. Tight-binding potentials for transition metals and alloys. *Physical Review B*, 48(1):22–33, 1993. doi: 10.1103/PhysRevB.48.22.

[23] William C. Swope, Hans C. Andersen, Peter H. Berens, and Kent R. Wilson. A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters. *The Journal of Chemical Physics*, 76(1):637–649, 1982. doi: 10.1063/1.442716.

[24] H. J. C. Berendsen, J. P. M. Postma, W. F. van Gunsteren, A. DiNola, and J. R. Haak. Molecular dynamics with coupling to an external bath. *The Journal of Chemical Physics*, 81(8):3684–3690, 1984. doi: 10.1063/1.448118.

[25] Benedict Leimkuhler and Charles Matthews. Rational construction of stochastic numerical methods for

molecular sampling. *Applied Mathematics Research eXpress*, 2013(1):34–56, 2013. doi: 10.1093/amrx/abs010.

[26] LAMMPS developers. LAMMPS benchmark input `bench/in.lj` and reference log `bench/log.15Jul25.lj.fixed.g++.1`. https://github.com/lammps/lammps/tree/develop/bench, . Accessed 2026-10-09.

[27] Loup Verlet. Computer "experiments" on classical fluids. II. equilibrium correlation functions. *Physical Review*, 165(1):201–214, 1968. doi: 10.1103/PhysRev.165.201.

[28] A. L. Mackay. A dense non-crystallographic packing of equal spheres. *Acta Crystallographica*, 15(9): 916–918, 1962. doi: 10.1107/S0365110X6200239X.

[29] Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, et al. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022. doi: 10.1109/TPDS.2021.3097283.

[30] LAMMPS developers. LAMMPS benchmarks (former `lammps.org/bench.html`), "billion-atom lj benchmarks". https://web.archive.org/web/20220422053012/https://www.lammps.org/bench.html, . Internet Archive snapshot of 2022-04-22, accessed 2026-10-09.